Experiment 4

Name : Astik Joshi **UID :** 23BCS10627
Branch: BE-CSE **Section/Group:** KRG-3B
Semester: 6th **Date of Performance:**-06/02/2026
Subject Name: Full Stack-II **Subject Code:** 23CSH-309

Aim: The aim of this implementation is To develop a secure navigation system using Context-based authentication, protected routing, and optimized React hooks to improve performance and UI using Material UI.

Objective:-

- To centrally handle authentication state across the entire application using Context API
- To configure client-side routing for seamless page navigation
- To safeguard protected routes by preventing access from unauthorized users
- To enable navigation between pages only after successful user authentication
- To automatically redirect users who are not logged in to the login page
- To enhance overall application security and user experience
- To ensure a clean, modular, and maintainable code architecture

Implementation/Code:

The **EcoTrack application** uses **Redux Toolkit** for centralized state management, ensuring predictable data flow and easier debugging. A global Redux store is configured using configureStore with feature-based slices like logsSlice, while asynchronous operations are handled using **Redux Async Thunks** to manage API request states. **React-Redux hooks** (useDispatch, useSelector) connect components to the store, enabling data fetching and state access. **Selectors** derive filtered views from the Redux state without mutation, and loading and error states are managed to improve performance and user experience.

- **/store/logSlice.js :-**

```
EcoTracker > src > store > JS logsSlice.js > ...
import { createSlice,createAsyncThunk } from "@reduxjs/toolkit";

export const fetchLogs = createAsyncThunk(
    "logs/fetchLogs",
    async()=>{
        await new Promise((resolve)=>setTimeout(resolve,1000));

        return [
  { id: 1, activity: "Car Travel", carbon: 4 },
  { id: 2, activity: "Electricity Usage", carbon: 7 },
  { id: 3, activity: "Cycling", carbon: 0 },
];
    }
)

const logsSlice = createSlice({
    name:"logs",
    initialState:{
        data :[],
        status:"idle",
        error:null,
    },
    reducers:{},
    extraReducers:(builder)=>{
        builder.addCase(fetchLogs.pending,(state)=>{
            state.status="loading";
        })
        .addCase(fetchLogs.fulfilled,(state,action)=>{
            state.status="succeeded";
            state.data=action.payload;
        })
        .addCase(fetchLogs.rejected,(state,action)=>{
            state.status="failed";
            state.error=action.error.message;
        })



    }
    })


export default logsSlice.reducer;
```

- **store.js :-**

```
import {configureStore} from '@reduxjs/toolkit';
import logsReducer from './logsSlice';
const store = configureStore({
    reducer: {
        logs: logsReducer,
    },
});
export default store;
```

- **LogSlice.jsx :-**

EcoTracker > src > JS logSlice.js > default

```javascript
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
export const fetchLogs = createAsyncThunk("logs/fetchLogs", async () => {
  return new Promise((resolve) =>
    setTimeout(() => resolve([
      { id: 1, activity: "Commute", carbon: 2.5 },
      { id: 2, activity: "Lunch", carbon: 0.5 },
    ]), 1000)
  );
});

const logSlice = createSlice({
  name: "logs",
  initialState: {
    data: [],
    status: "idle", // 'idle' | 'loading' | 'succeeded' | 'failed'
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchLogs.pending, (state) => {
        state.status = "loading";
      })
      .addCase(fetchLogs.fulfilled, (state, action) => {
        state.status = "succeeded";
        state.data = action.payload;
      })
      .addCase(fetchLogs.rejected, (state, action) => {
        state.status = "failed";
        state.error = action.error.message;
      });
  },
});

export default logSlice.reducer;
```

- **App.jsx:-**

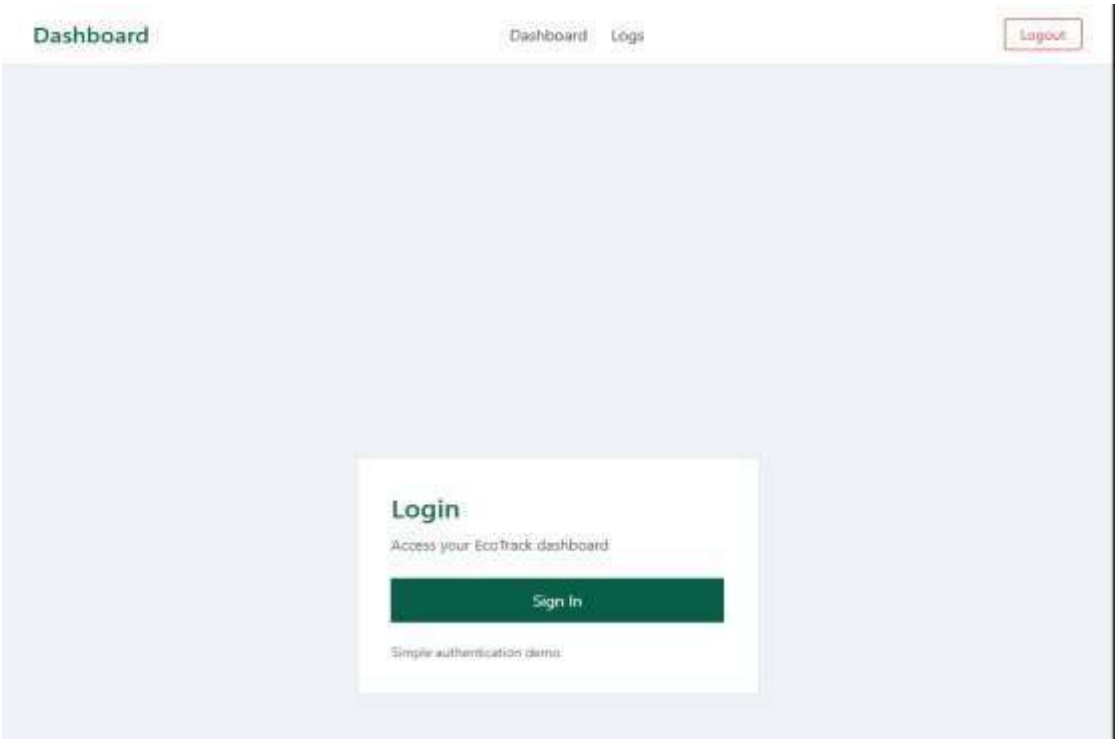
EcoTracker > src > App.jsx > App

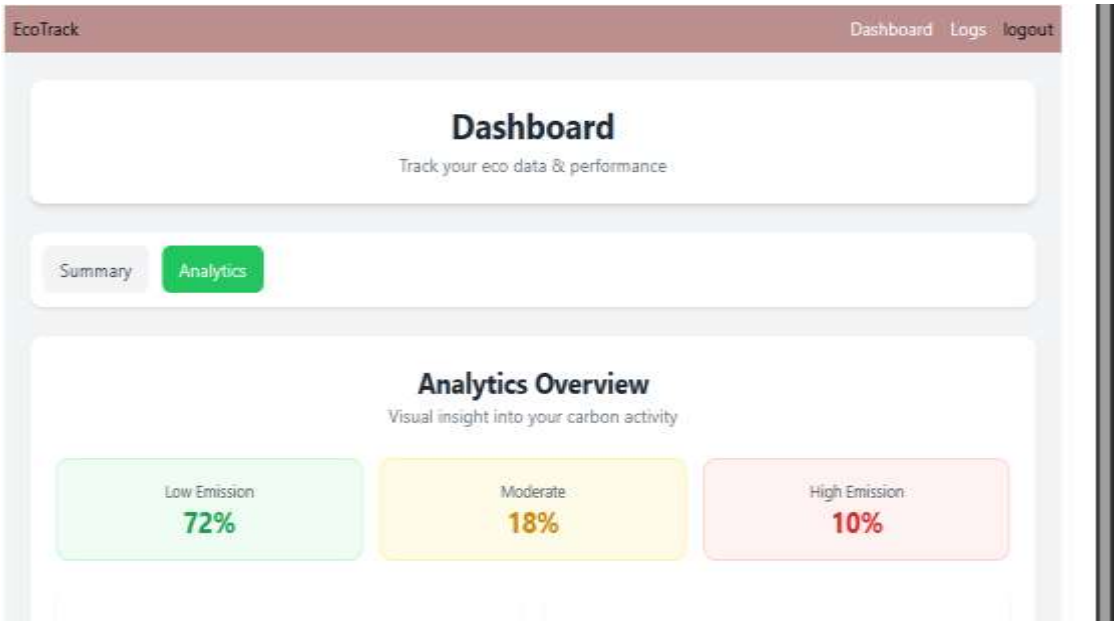
```jsx
import './App.css'
import Header from './components/Header'
import Dashboard from './pages/Dashboard.jsx'
import { BrowserRouter,Route,Routes } from 'react-router-dom'
import ProtectedRoute from './routes/ProtectedRoute'
import DashboardSummary from './pages/DashboardSummary'
import DashboardAnalytics from './pages/DashboardAnalytics'
import Login from './pages/Login'
import Logs from './pages/Logs'

function App() {

  return (
    <main>
    <>
    <BrowserRouter>
    <Header  title={"EcoTrack"}/>

     <Routes>

      <Route path="/login" element={<Login/>} />

      <Route path="/" element={
        <ProtectedRoute>
          <Dashboard/>
        </ProtectedRoute>
      }>


      <Route index element={<DashboardSummary/>} />
      <Route path="summary" element={<DashboardSummary/>} />
      <Route path="analytics" element={<DashboardAnalytics/>} />

      </Route>
      <Route path="logs" element={<Logs/>} />


    </Routes>

    </BrowserRouter>
    </>
    </main>

  )
}

export default App
```

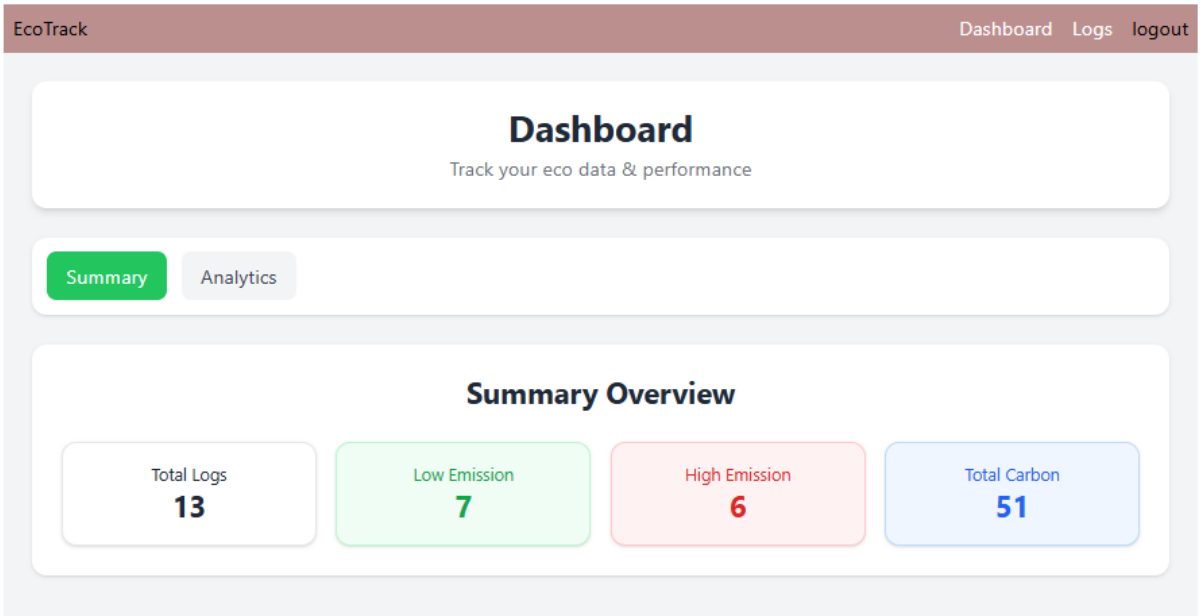
- ProtectRoute.jsx

```jsx
import {Navigate} from "react-router-dom";
import {useAuth} from "../context/AuthContext";

const ProtectedRoute = ({children}) => {
    const {isAuthenticated} = useAuth();

    if(!isAuthenticated){
        return <Navigate to="/login" replace />;
    }
    return children;
}
export default ProtectedRoute;
```

4. Output




EcoTrack
Dashboard
Logs
logout
Dashboard
Track your eco data & performance
Summary
Analytics
Analytics Overview
Visual insight into your carbon activity
Low Emission
72%
Moderate
18%
High Emission
10%


EcoTrack
Dashboard
Logs
logout
Dashboard
Track your eco data & performance
Summary
Analytics
Summary Overview
Total Logs
13
Low Emission
7
High Emission
6
Total Carbon
51

5. Learning Outcome :-

- Learned performance optimization using React.memo
- Implemented secure routing with context API
- Built modern UI using Material UI
- Improved scalability and maintainibility